

Ejercicio 1 - Práctica 2

1. Single Responsibility Principle (SRP) — Principio de Responsabilidad Única

Una clase debe tener una, y solo una, responsabilidad. El principio de responsabilidad única dice que una clase o componente de software debe ocuparse de una sola funcionalidad. Cuando una clase toma múltiples responsabilidades no relacionadas (como por ejemplo lógica de negocio y persistencia en base de datos) cualquier modificación en una puede introducir efectos colaterales indeseados en la otra.

Ejemplo: Incumplimiento de SRP

La clase Factura calcula el total de la venta y también gestiona cómo se guarda la factura en la base de datos.

```
public class Factura {  
    private double monto;  
  
    public Factura(double monto) {  
        this.monto = monto;  
    }  
  
    // Responsabilidad 1  
    public double calcularTotalConImpuesto() {  
        return this.monto * 1.21;  
    }  
  
    // Responsabilidad 2  
    public void guardarEnBaseDeDatos() {  
        System.out.println("Guardando la factura en la base de datos SQL...");  
    }  
}
```

Problema: Si cambia la estructura de la base de datos hay que modificar factura o si cambia la fórmula también hay que modificar la clase. Tiene dos razones distintas para cambiar.

Solución: Separar las responsabilidades en dos clases distintas una para la lógica de la entidad y otra para el almacenamiento.

```
public class Factura {  
    1 public class Factura {  
    2     private double monto;  
    3  
    4     public Factura(double monto) {  
    5         this.monto = monto;  
    6     }  
    7  
    8     public double calcularTotalConImpuesto() {  
    9         return this.monto * 1.21;  
   10     }  
   11  
   12     public double getMonto() {  
   13         return monto;  
   14     }  
   15 }  
   16  
   17 public class FacturaRepository {  
   18     public void guardar(Factura factura) {  
   19         System.out.println("Guardando factura con monto " + factura.getMonto() + " en base de datos.");  
   20     }  
   21 }
```

Open Closed Principle (OCP) — Principio de Abierto para la extensión y cerrado para la modificación.

El principio establece que debemos ser capaces de incorporar nuevas funcionalidades o comportamientos a una clase sin necesidad de modificar el código fuente original ya probado y testeado. Esto se logra mediante la abstracción (interfaces o clases abstractas) y el polimorfismo.

Ejemplo: Incumplimiento de OCP

El procesador de pagos utiliza un bloque condicional para determinar el flujo de cobro según el tipo de pago.

```
public class ProcesadorDePagos {  
    1 public class ProcesadorDePagos {  
    2  
    3     public void procesarPago(String metodo, double monto) {  
    4         if (metodo.equalsIgnoreCase("TARJETA")) {  
    5             System.out.println("Cobrando $" + monto + " con Tarjeta de Crédito.");  
    6         } else if (metodo.equalsIgnoreCase("PAYPAL")) {  
    7             System.out.println("Cobrando $" + monto + " mediante cuenta PayPal.");  
    8         }  
    9         // Si queremos agregar un nuevo sistema de pags debemos modificar la clase.  
   10     }  
   11 }
```

Solución Aplicando OCP: Definimos una abstracción que encapsula la operación. Los nuevos métodos de pago solo implementan la interfaz sin alterar el código.

```
public interface MetodoPago {  
    1 public interface MetodoPago {  
    2     void pagar(double monto);  
    3 }  
    4  
    5 public class PagoTarjeta implements MetodoPago {  
    6     @Override  
    7     public void pagar(double monto) {  
    8         System.out.println("Cobrando $" + monto + " con Tarjeta de Crédito.");  
    9     }  
   10 }  
   11  
   12 public class PagoPaypal implements MetodoPago {  
   13     @Override  
   14     public void pagar(double monto) {  
   15         System.out.println("Cobrando $" + monto + " mediante cuenta PayPal.");  
   16     }  
   17 }  
   18  
   19 public class ProcesadorDePagos {  
   20     public void procesar(MetodoPago metodoPago, double monto) {  
   21         metodoPago.pagar(monto);  
   22     }  
   23 }
```

2. Patrón de Diseño Singleton

El patrón Singleton es un patrón creacional que busca garantizar que una clase tenga únicamente una instancia en memoria a lo largo de todo el ciclo de vida de la aplicación, dando un punto de acceso global a ella.

El patrón Singleton no cumple SRP debido a que asume simultáneamente dos responsabilidades fundamentales no relacionadas:

La gestión del ciclo de vida y control de instanciación ya que es responsable de asegurar que exista una sola instancia de sí mismo, crearla si no existe y restringir el acceso a su constructor.

La lógica de negocio porque además de autogestionarse debe llevar a cabo su rol primario dentro del dominio de la aplicación por ejemplo gestionar conexiones a base de datos o administrar configuraciones del sistema.

El patrón Singleton trae dificultades al momento de crear pruebas unitarias por varias causas, para empezar al existir una única instancia compartida por toda la suite de pruebas, el estado interno del Singleton persiste entre un test y otro. Esto rompe la independencia de los tests. Por otro lado la imposibilidad de simulación ya que el acceso se realiza a través de un método estático global y el constructor suele ser privado, los frameworks de testing convencionales se ven obligados a realizar pruebas integradas en vez de unitarias puras. También surgen problemas con el “Acoplamiento invisible” las clases que utilizan el Singleton no declaran explícitamente sus dependencias en el constructor; las invocan internamente mediante llamadas estáticas. Esto oscurece las dependencias reales del componente frente a quien escribe la prueba. Y por último problemas de paralelismo si los tests se ejecutan en paralelo en múltiples hilos, los accesos simultáneos y concurrentes al Singleton pueden generar condiciones de carrera adulteradas.

3. Dependency Injection Design Pattern

Es un patrón de diseño que externaliza la creación de las dependencias de una clase. En lugar de que un objeto instancie directamente los servicios o colaboradores que necesita para operar esas dependencias le son suministradas desde el exterior (generalmente a través del constructor, métodos setter o un contenedor de inversión de control).

Su objetivo principal es desacoplar a las clases de las implementaciones concretas de sus dependencias.

Diferencia entre Dependency Inversion y Dependency Injection

Aunque se complementan en la práctica representan conceptos en niveles de abstracción diferentes:

Aspecto	Dependency Inversion Principle (DIP)	Dependency Injection (DI)
Categoría	Principio de diseño	Patrón de diseño estructural/creacional.
Nivel	Nivel conceptual / arquitectónico de alto orden.	Nivel táctico / técnico de implementación.
Definición	Establece que los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (interfaces). Además, las abstracciones no deben depender de los detalles, sino los detalles de las abstracciones.	Es el mecanismo concreto mediante el cual se le entregan las dependencias a un objeto cliente desde afuera, sin que este deba construirlas por su cuenta.
Relación	Dicta la regla de diseño : Depende de interfaces, no de clases concretas.	Es una de las herramientas técnicas para llevar a cabo el principio en la práctica.

Ejemplo sin Inyección de Dependencias (Acoplamiento fuerte)

En este caso `NotificadorUsuario` crea directamente una instancia concreta de `ServicioEmail` por lo que no es posible cambiar el canal de comunicación sin modificar la clase ni testearla sin enviar un email real.

```
public class ServicioEmail {
1   public class ServicioEmail {
2       public void enviar(String mensaje) {
3           System.out.println("Email enviado: " + mensaje);
4       }
5   }
6
7   public class NotificadorUsuario {
8       private ServicioEmail servicioEmail;
9
10      public NotificadorUsuario() {
11          this.servicioEmail = new ServicioEmail();
12      }
13
14      public void notificar(String mensaje) {
15          this.servicioEmail.enviar(mensaje);
16      }
17  }
```

Ejemplo con Inyección de Dependencias (Por Constructor) y DIP

Aplicamos DIP creando una interfaz `ServicioMensajeria` e inyectamos la implementación requerida a través del constructor.

```
public interface ServicioMensajeria {
1   void enviarMensaje(String destinatario, String contenido);
2   }
3
4   public class EmailService implements ServicioMensajeria {
5       @Override
6       public void enviarMensaje(String destinatario, String contenido) {
7           System.out.println("Enviando Email a " + destinatario + ": " + contenido);
8       }
9   }
10
11   public class SMSService implements ServicioMensajeria {
12       @Override
13       public void enviarMensaje(String destinatario, String contenido) {
14           System.out.println("Enviando SMS al número " + destinatario + ": " + contenido);
15       }
16   }
17
18   public class NotificadorUsuario {
19       private final ServicioMensajeria servicioMensajeria;
20
21       public NotificadorUsuario(ServicioMensajeria servicioMensajeria) {
22           this.servicioMensajeria = servicioMensajeria;
23       }
24
25       public void registrarYNotificar(String usuario, String mensaje) {
26           servicioMensajeria.enviarMensaje(usuario, mensaje);
27       }
28   }
29
30   public class Main {
31       public static void main(String[] args) {
32           ServicioMensajeria email = new EmailService();
33           NotificadorUsuario notificadorEmail = new NotificadorUsuario(email);
34           notificadorEmail.registrarYNotificar("juan@empresa.com", "Bienvenido a la plataforma");
35
36           ServicioMensajeria sms = new SMSService();
37           NotificadorUsuario notificadorSMS = new NotificadorUsuario(sms);
38           notificadorSMS.registrarYNotificar("+5491122334455", "Su código de acceso es 4821");
39       }
40   }
41 }
```